The TX Path

How the operator's voice gets from the page into Node Talker, through the key-up checks, the transmit buffer and the pacer, and out to Asterisk as USRP packets on the transmit node.

KEY-UP | BUFFER | USRP | NODE 1921

SUMMARY · READ-ONLY REVIEW

Node 1921 in rpt.conf

Transmit needs its own node in Asterisk. Node Talker writes it into rpt.conf, and then links it to the main node so its audio reaches the main node's transmitter.

1.1 The block it writes

The setup adds a comment and a node block for node 1921. The block uses a USRP channel that listens on 127.0.0.1:34018 and sends to 127.0.0.1:32018. It is set to half duplex, with no hang time, no alternate hang time, telemetry off by default, no courtesy tone on unkey, and it sets an identification recording on a timer. The block inherits the templates of the main node, so it shares that node's settings.

The setup also adds a line to the `[nodes]` section of rpt.conf, so that Asterisk can find the node: `1921 = radio@127.0.0.1/1921,NONE`. If iax.conf sets a bind port other than 4569, the port is added after the address. The section is created if it is missing.

1.2 Taking over from Node Recorder

If node 1921 already exists as a Node Recorder block, the setup takes it over, so that recording continues from this node. If a node 1921 exists that Node Talker did not write, the setup refuses and asks the operator to remove it or pick another.

1.3 Asterisk restart and Allmon3

After the file is written, Asterisk must be restarted before the node takes effect. A backup of rpt.conf is kept with each change. The setup also adds node 1921 to the Allmon3 configuration (`allmon3.ini`) so that the node shows there. The result tells the operator to restart Allmon3.

1.4 Readiness

The transmit path is ready when the node 1921 block exists with the right channel, the main node is set, and the `[nodes]` line includes node 1921. The page shows the state from this check, and the link is reported as linked or not linked from the main node's current links.

The link keeper

A background loop checks every 20 seconds. The link state comes from the Asterisk command `rpt lstats` for the main node. If the transmit path is ready and node 1921 is not linked, the loop links it with `rpt cmd <main> ilink 3 1921` and writes a log line. The loop logs the first failure, and logs again when the link comes back. When the link is up, the loop does nothing.

Unlinking happens only when the transmit path is turned off or Node Talker stops. Ending a transmission does not unlink the node.

Checks before the node keys

When the page asks to key, the server runs these checks in order:

1. **Ready.** The transmit path must be ready (Part 1.4). If not, the key is refused as "not-ready".
2. **Linked.** Node 1921 must be linked to the main node. If not, the key is refused as "not-linked".
3. **Busy.** With `busy_lockout` on, a key is refused as "busy" while the channel is active (Part 2 of the RX series), unless Node Talker itself unkeyed less than one second ago. This stops a key-up over someone else's transmission.
4. **In use.** Only one operator can hold the key. A second operator is refused as "in-use".

A refused key gives the operator a short message, for example "The node is busy (someone is talking)."

When the key succeeds, the buffers are cleared and a new session starts. If phone tones are on (Part 6), the phone is opened as well.

From the page into the buffer

The page opens a WebSocket at `/api/talk`. The route needs a login. The request must be a proper upgrade, and its `Origin` must be the page's own host over https or http, or it is refused with 403 ("Wrong origin"). The server pings every 20 seconds and drops the socket after 60 seconds without traffic. When the socket closes for any reason, an active transmission from it ends.

1. The page sends microphone audio as binary WebSocket messages. Each message is 16-bit PCM. The page sends the key and unkey requests as text messages.
2. The server accepts audio only from the operator who holds the key. Audio from anyone else is ignored.
3. The server cuts the audio into 320-byte frames (20 ms). Each frame gets the transmit gain (`tx_gain_db`, when the level settings are active) and its peak level is noted.
4. Frames wait in a queue. The queue holds up to 50 frames (1 s) for a normal transmission, or 300 frames (6 s) for a phone session.

If the queue passes its limit, the oldest frames are dropped.

Sending at a steady pace

A paced loop runs every 20 ms and decides what to send for that tick. The key rules are:

STAGE	WHAT HAPPENS
Start	Nothing is sent until the queue holds the transmit buffer (`tx_buffer_ms`, default 120 ms, or six frames). If the buffer never fills, sending starts once the fill time has passed.
Sending	One frame is sent for each tick. A fade-in is applied after a gap.
Gap	If the queue runs dry while sending, the pacer fades out the last frame and counts a gap. It then waits for the buffer to refill before it sends again.
Release	When the operator lets go, the pacer sends what is left, fades the last frame out, and then stops.
Time limit	After `tot_sec` (default 120 s, from 30 to 600), the transmission ends with the reason "timeout".
Deadman	If no audio arrives for one second, the transmission ends with the reason "deadman".

The gap count is part of the transmission state and the log. A transmission with gaps is logged as a warning.

USRP packets

6.1 The packets

Each frame goes to Asterisk as one UDP packet. The packet starts with a 32-byte USRP header, followed by the 320 bytes of audio. The header carries a sequence number that rises with each packet, and a key bit. The key bit is set on the packets that carry audio, and is cleared on the packets that end the transmission.

6.2 Ending a transmission

When the transmission ends, three silent packets are sent with the key bit cleared. Asterisk then drops its transmitter.

6.3 The drain loop

The transmit node also sends its own output to 127.0.0.1:34018, and a drain loop listens there. The loop forwards every datagram to the Node Recorder's forwarding port, which is how this channel's audio reaches Node Recorder. If the port cannot be opened, the loop logs a warning and tries again every 60 seconds.

Opening the far end of a phone call

Node Talker does not send phone tones itself. When the node is in a phone call, it asks the Dashboard (Node Controller) to send them, through the Dashboard's local API (loopback only, with the local key).

1. **Check the call.** On a key-up with phone tones on (`phone_tones`, on by default), a background worker asks the Dashboard for the phone status. Tones are used only when the state is "in call" and the call type is one of sip, sip_ip, hoip, awire or iax2. Otherwise nothing is sent and the transmission is not held.
2. **Open.** The worker asks the Dashboard to send the open code (`*99` by default). The transmission waits for this, for at most 5 seconds. After a good send it keeps waiting for the lead time (`phone_lead_ms`, default 500 ms) before audio starts. If the send fails, the transmission goes ahead anyway and the page is told.
3. **Close.** When the transmission ends, and only if the open code was sent, the worker asks the Dashboard to send the close code (`#` by default).

A phone session also gets the larger queue (300 frames, 6 seconds) instead of 50. If the Dashboard is not running, the status says "Node Controller is not running", and no tones are sent. If the Dashboard is too old to know the phone actions, the status says it needs 9.3.104-pi02w (pi02w build) or 9.3.99 (full build) or later.

What the TX path shows

8.1 Two checks stop a bad key

The link check and the busy check stop most bad transmissions before the node keys. The busy check is the one operators need to understand: a refused key on a busy channel is the system doing its job, not a fault.

8.2 Ending is always clean

Every ending, whether release, timeout, deadman or a dropped connection, sends the same silent packets and writes a log line with its reason.

8.3 Not checked

- The Asterisk side of the USRP channel on a running node.
- The phone protocol path in full (Part 7).
- How the link keeper behaves when Asterisk is restarting.